

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 04 -

VULNERABILIDADES CRÍTICAS: OWASP E CVES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	11
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13

O QUE VEM POR AÍ?

Em arquiteturas *cloud-native*, a segurança não pode ser um gargalo no final do processo. A resposta está em integrar a segurança proativa, ou *Shift Left*, no DNA da entrega contínua.

Nesta aula, você mergulhará nos fundamentos que transformam a segurança em código. Você aprenderá a fortalecer as unidades de *deployment* (containers) na origem e a auditar o ambiente onde elas irão rodar (IaC), garantindo que apenas artefatos blindados e infraestruturas conformes cheguem à produção. Prepare-se para automatizar a prevenção de vulnerabilidades e a governança de conformidade com padrões de mercado.

HANDS ON

A teoria se materializa em nosso ambiente de codificação, onde os princípios arquiteturais que acabamos de abordar serão implementados diretamente no seu pipeline de CI/CD. O material prático servirá como seu guia para transformar filosofia em funcionalidade, utilizando código real disponível no repositório do Github. Na primeira etapa, focaremos no **Hardening de Dockerfiles**, aplicando o Princípio do Menor Privilégio na prática, onde você verá como refatorar um Dockerfile vulnerável para utilizar usuários não-root e imagens minimalistas, como as *distroless*, reduzindo drasticamente a superfície de ataque.

Em seguida, dedicaremos nossa atenção à **Varredura de Imagens**, integrando o scanner Trivy diretamente no fluxo de *build*. A prática demonstrará como configurar o pipeline para que a varredura se torne um *quality gate*, impedindo que imagens contendo vulnerabilidades conhecidas chegassem ao AWS ECR (Elastic Container Registry), aplicando assim o conceito de Segurança Imutável.

Por fim, a prática técnica abordará a **Segurança da Infraestrutura como Código (IaC)**, onde o Checkov será usado para auditar arquivos Terraform. Você configurará verificações (checks) para identificar e falhar o *pipeline* em casos de *misconfigurations* comuns e de alto risco, como buckets S3 públicos, demonstrando a implementação efetiva do Policy as Code antes mesmo que o *provisioning* da infraestrutura seja iniciado.

SAIBA MAIS

Segurança Inerente: Os Fundamentos Arquiteturais de DevSecOps

A evolução da engenharia de software exige que a segurança seja tratada como uma característica de qualidade inegociável. Para o engenheiro sênior, essa mudança de paradigma é suportada por pilares teóricos e ferramentas que permitem a automação da conformidade e da contenção de riscos na nuvem. A seguir, detalharemos os fundamentos que sustentam uma arquitetura DevSecOps resiliente.

O Princípio do Menor Privilégio (PoLP) e a Contenção de Riscos em Containers

O Princípio do Menor Privilégio (PoLP) é a pedra angular da segurança zero trust. Sua premissa é simples, mas essencial: qualquer usuário, aplicação ou processo deve ter apenas as permissões estritamente necessárias para realizar sua tarefa, e nada mais. No universo dos containers, a falha em implementar o PoLP é a causa direta de grande parte dos riscos de segurança em tempo de execução. Se um contêiner for comprometido, o PoLP impede que o atacante propague *malware* ou escale privilégios para o *host* subjacente.

A principal falha é a execução do processo principal do contêiner como o usuário root (UID 0), o que cria um vetor de ataque catastrófico, dando acesso irrestrito ao invasor. A execução não-root, via instrução USER no Dockerfile, é obrigatória para limitar o alcance de uma exploração. Para garantir a eficácia do PoLP, o engenheiro deve garantir que o usuário não-root tenha as permissões de sistema de arquivos adequadas para leitura/escrita e que todos os binários sejam de propriedade do root e não sejam graváveis pelo usuário da aplicação, reforçando a imutabilidade do artefato.

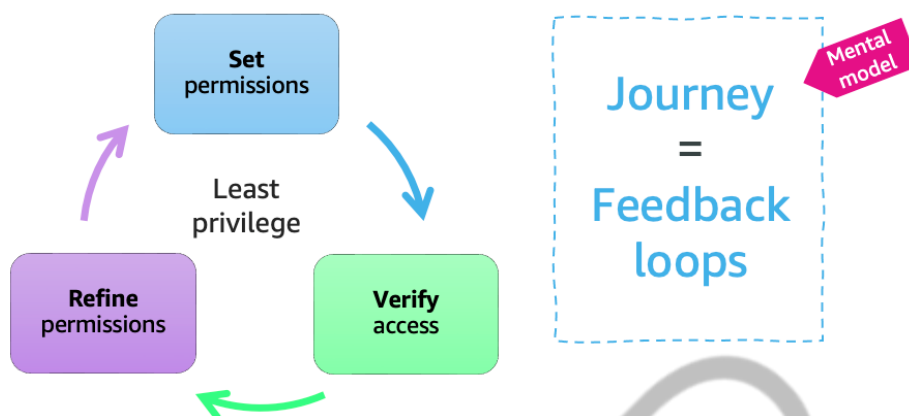


Figura 1 – Princípio do Menor Privilégio em Containers (PoLP)
Fonte: [Cloudfront](#) (2025)

Segurança Imutável: O Paradigma "Build Once, Run Forever"

O conceito de Segurança Imutável se baseia na filosofia *Build Once, Run Forever*. A premissa é que um contêiner ou uma instância de infraestrutura, após ser implantada, nunca deve ser modificada. Qualquer alteração, seja uma atualização de código ou um *patch* de segurança, exige a construção de um novo artefato, que deve ser validado e implantado novamente, substituindo a versão antiga.

Essa abordagem é a solução arquitetural para o problema do *Configuration Drift* (desvio de configuração), que ocorre por intervenções manuais, introduzindo vulnerabilidades não rastreadas. A imutabilidade aumenta a resiliência contra ataques de persistência. Se um contêiner for comprometido, a substituição ou reinicialização realizada pelo orquestrador o retornará a unidade à sua imagem base segura, minimizando o tempo de exposição. Isso exige que o *hardening* e a varredura sejam processos obrigatórios na fase de CI, atuando antes mesmo do *push* para o registro.

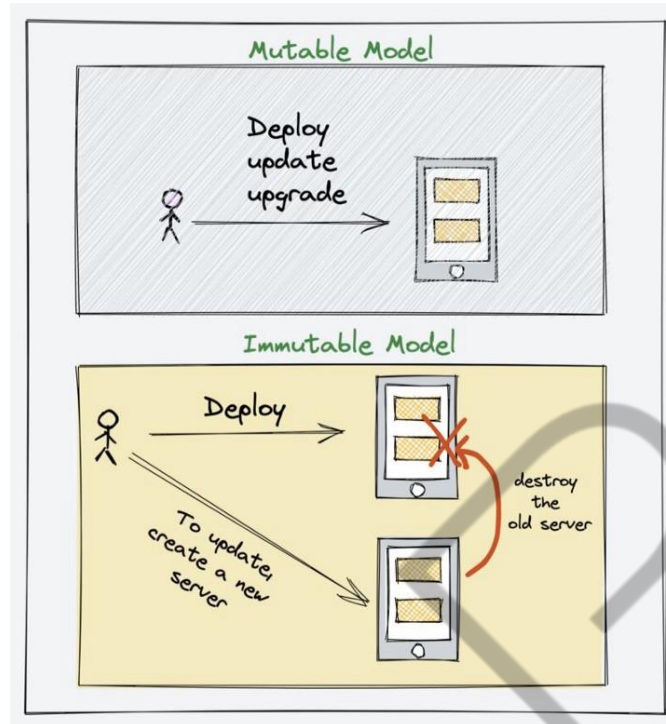


Figura 2 – Imutabilidade de Infraestrutura
Fonte: [Devopscube](#) (2025)

Hardening de Container Images: Da Base Mínima ao CIS Benchmark

O *hardening* de imagens é a aplicação sistemática dos princípios de PoLP e Imutabilidade para reduzir drasticamente a superfície de ataque, seguindo padrões de segurança reconhecidos. O principal padrão para esta prática é o **CIS Docker Benchmark**, que define configurações seguras recomendadas para todo o ecossistema Docker, auxiliando na conformidade regulatória.

As estratégias de *hardening* visam otimizar a imagem para segurança e performance:

1. **Imagens Base Minimalistas:** o uso de bases pesadas (como ubuntu), introduz binários e ferramentas desnecessárias, aumentando a superfície de ataque. As opções preferenciais são as **Imagens Distroless**, da Google, que contêm apenas o *runtime* essencial, tornando a exploração interativa significativamente mais difícil pela ausência de um *shell* ou gerenciadores de pacotes.
2. **Multi-Stage Builds (Construção Multi-Etapas):** esta prática isola o ambiente de construção, que é pesado, do ambiente final de produção,

que é minimalista e seguro. Ao copiar apenas o binário final, *caches* de pacotes, dependências de desenvolvimento e segredos de tempo de *build* são removidos, resultando em uma imagem menor, mais rápida de implantar e com menor superfície de ataque.

3. **Limpeza de Layers:** para garantir a eficiência do *layer* caching e minimizar o tamanho da imagem, comandos de instalação e limpeza de arquivos temporários devem ser combinados em uma única instrução RUN no Dockerfile.

Princípio de Hardening	Instrução Chave	Exemplo de Implementação	Justificativa de Segurança
Menor Privilégio	USER <non-root-user>	RUN adduser -D appuser && USER appuser	Impede o escalonamento de privilégios e limita o alcance de uma potencial exploração.
Redução da Superfície	Imagens base mínimas	FROM gcr.io/distroless/static-debian10	Elimina <i>shells</i> , gerenciadores de pacotes e binários desnecessários, reduzindo vulnerabilidades.
Otimização/Segurança	Limpeza de caches	RUN apt-get update && apt-get install -y && rm -rf /var/lib/apt/lists/*	Reduz o tamanho da imagem (<i>layer</i> cache) e remove dados temporários potencialmente sensíveis.
Isolamento de Build	Multi-Stage Builds	COPY --from=builder /app/binary /usr/bin/	Garante que dependências de desenvolvimento e segredos de tempo de <i>build</i> não contaminem o ambiente de <i>runtime</i> .

Tabela 1 – Estratégias de Hardening em Dockerfiles
Fonte: Elaborado pelo autor (2025)

Varredura Contínua: Integrando Trivy e AWS ECR no Pipeline

O *hardening* é complementado pela varredura automatizada e contínua para detectar vulnerabilidades que surgem de bibliotecas e dependências de terceiros. **Trivy**, uma ferramenta de código aberto mantida pela Aqua Security, é a solução preferida no ecossistema *cloud-native* por sua versatilidade *all-in-one*, permitindo escanear imagens de contêiner, IaC e segredos expostos.

A integração do Trivy no pipeline de CI/CD (o *quality gate*) materializa a filosofia *Shift Left*. Se vulnerabilidades com severidade crítica ou alta forem detectadas, o processo de *build* deve falhar explicitamente, impedindo que a imagem comprometida seja publicada. A varredura *on push* no AWS ECR (Elastic Container Registry) complementa o Trivy, atuando como uma rede de segurança secundária e um mecanismo de monitoramento contínuo. É essencial que a integração utilize **Credenciais de Curta Duração**, assumindo *IAM Roles* ou *Service Accounts* (via AWS SDK/OIDC), garantindo que chaves de acesso estáticas e de longa duração não sejam armazenadas.

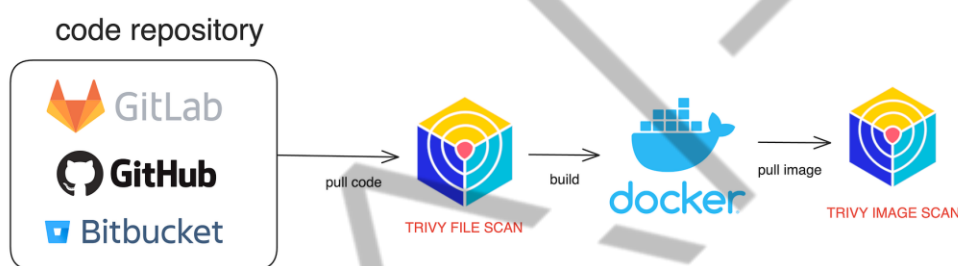


Figura 3 – Trivy em Pipeline CI/CD
Fonte: [Medium](#) (2025)

Policy as Code (PaC) e a Auditoria de Infraestrutura

Policy as Code (PaC) é a prática de codificar políticas de segurança e conformidade para verificar a Infraestrutura como Código (IaC). O PaC atua como uma camada essencial de análise estática (SAST) voltada para a infraestrutura, permitindo que a governança seja imposta de maneira automatizada e consistente, prevenindo o *misconfiguration* antes do *provisioning*.

Checkov é a ferramenta líder de código aberto para varredura de *misconfigurations* em IaC, suportando Terraform, CloudFormation e Kubernetes. Ele oferece mais de 750 políticas pré-definidas que avaliam o código contra padrões de conformidade importantes, como CIS e AWS Well-Architected Framework. A integração do Checkov no CI/CD mitiga o risco de provisionar recursos com alto potencial de exposição. Por exemplo, o Checkov audita definições de *buckets* AWS

S3. Se um arquivo Terraform configurar um *bucket* com permissões excessivamente permissivas (como *public-read*), o Checkov irá falhar a varredura, impedindo que o comando de *apply* seja executado. O Checkov pode, inclusive, escanear o resultado final interpolado do plano de execução do Terraform (*terraform plan*), garantindo que todas as variáveis sigam a política de conformidade.

Aprofundamento: AWS EKS Security Best Practices (Pod Security Standards - PSS)

Para ambientes de orquestração complexos, como o AWS EKS (Amazon Elastic Kubernetes Service), a segurança dos *pods* é a última linha de defesa para aplicar o PoLP em *runtime*. As tradicionais *Pod Security Policies* (PSPs) foram substituídas pelo **Pod Security Admission (PSA)**, um controlador de admissão nativo do Kubernetes que impõe os **Pod Security Standards (PSS)**.

O PSS define três perfis de segurança progressivos. O perfil **Baseline** é minimamente restritivo, projetado para prevenir escalonamentos de privilégios conhecidos e que a maioria das aplicações deve operar. O perfil **Restricted** é o mais rigoroso, exigindo que o *pod* seja executado como um usuário não-root (ou com um ID de usuário alto) e restringe o uso de certos tipos de volumes, sendo o padrão ouro para aplicações sensíveis. O perfil **Privileged** é irrestrito e reservado apenas para *workloads* de infraestrutura confiáveis, como agentes de monitoramento que necessitam de acesso privilegiado ao nó. A aplicação do PSS via PSA no AWS EKS garante que, mesmo que o pipeline de CI/CD tenha falhado em detectar uma falha no Dockerfile, o controlador de admissão do Kubernetes rejeitará o *deployment* se o *namespace* estiver configurado para impor um perfil restritivo.

MERCADO, CASES E TENDÊNCIAS

Integração da Segurança no Pipeline de Desenvolvimento, conhecida como DevSecOps, transformou-se em um requisito fundamental para a resiliência de negócios. A complexidade crescente das arquiteturas *cloud-native* e a sofisticação das ameaças cibernéticas forçaram as organizações a abandonarem as abordagens de segurança reativas e baseadas em perímetros. A filosofia DevSecOps, que exige a integração da segurança desde o início do ciclo de vida (*Shift Left*), é hoje vista como o padrão ouro no desenvolvimento de aplicações.

A análise de grandes violações de segurança demonstra o custo qualitativo de não adotar uma cultura DevSecOps proativa. Incidentes recentes em grandes corporações de energia e serviços de saúde demonstram que abordagens de segurança reativas não conseguem acompanhar a velocidade do desenvolvimento moderno, resultando em vulnerabilidades críticas na cadeia de suprimentos de software. Esses casos forçaram a reconstrução de redes inteiras e expuseram falhas de governança. Em contraste, empresas que alcançaram a maturidade DevSecOps demonstram maior consistência em *deployments* e maior conformidade regulatória, transformando a segurança em um facilitador para a agilidade.

A tendência mais significativa na governança de ambientes de nuvem é a ascensão do **Policy as Code (PaC)**. À medida que as organizações operam em ambientes multi-nuvem, a capacidade de padronizar e impor políticas de segurança e conformidade de forma consistente e automatizada torna-se inviável sem a codificação da governança. O PaC, executado por ferramentas como Checkov, minimiza o risco de *misconfiguration* causado por erro humano, aplicando as regras de governança de forma não negociável em estágios iniciais do desenvolvimento.

O QUE VOCÊ VIU NESTA AULA?

Esta aula consolidou o conceito de segurança como código (DevSecOps), com ênfase no Princípio do Menor Privilégio e na Imutabilidade.

Os pontos centrais da prática foram:

Hardening de Containers: redução da superfície de ataque com Multi-Stage Builds e imagens minimalistas (Distroless), seguindo o CIS Docker Benchmark.

Varredura Contínua: uso do scanner Trivy no pipeline (Quality Gate) para detecção proativa de vulnerabilidades em imagens.

Governança de IaC: implementação de Policy as Code com Checkov para auditar configurações Terraform e CloudFormation e prevenir *misconfigurations*.

Segurança em EKS: análise dos Pod Security Standards (PSS) do Kubernetes como camada final de defesa em orquestração.

REFERÊNCIAS

CYBERARK. **Learn More About the Principle of Least Privilege.** 2025. Disponível em: <https://www.cyberark.com/what-is/least-privilege/>. Acesso em: 28 nov. 2025.

ILLUMIO. **DevSecOps.** 2025. Disponível em: <https://www.illumio.com/pt-br/cybersecurity-101/devsecops>. Acesso em: 28 nov. 2025.

KUBERNETES. **Pod Security Standards.** 2025. Disponível em: <https://kubernetes.io/docs/concepts/security/pod-security-standards/>. Acesso em: 28 nov. 2025.

SYSDIG. **Dockerfile Best Practices.** 2025. Disponível em: <https://www.sysdig.com/learn-cloud-native/dockerfile-best-practices>. Acesso em: 28 nov. 2025.

TRIVY. **The All-in-One Security Scanner.** 2025. Disponível em: <https://trivy.dev/>. Acesso em: 28 nov. 2025.

PALAVRAS-CHAVE

DevSecOps. Hardening. Containers. IaC. Menor Privilégio. Imutabilidade. Policy as Code. Trivy. Checkov. EKS. PSS. CIS Benchmark. Dockerfile. Varredura. CloudFormation.

EMANIP



POS TECH